n = 83 K899: the nested 899-column factor base under the rank-probe producer, end-to-end cold

2026-10-19

n = 83 K899: the nested 899-column factor base under the rank-probe producer, end-to-end cold

Status (2026-10-19): protocol frozen; Tier 1 run started on branch `claude/n83-k899`; no result yet. The protocol (frozen 2026-10-19T18:48:09Z) and the schedule were committed before any Tier 1 timing. The go/no-go diagnostic ran on a GitHub runner (run 39292460576, archived in `diag/run_github_39292460576/`). It is one process per arm with one rank seed, on Q0. It is a diagnostic, not timing evidence. Numbers from the cloud-agent VM are labelled **local**.

Question

The n = 83 a = 1 one-target index-calculus baseline is the rank-probe producer, `koblitz-n83-rankprobe-20261019` (PR #980, run 39214027289). It runs at K600: end-to-end cold 166.0 s, of which the rank stage is 143.7 s. Its rank stage makes 600 attempts. Each attempt probes the root table until the first hit, and the mean number of probes follows the law r / (166 E), where E ≈ 83 K² root-table entries. So K moves two terms in opposite directions:

- the rank stage, K x r / (166 x 83 K²) probes, which falls like 1/K;
- the index build and its self-tests, which grow like K².

A pre-registration cost model put the optimum near K ≈ 900: E2E(K) ≈ 146.9/a + 21.6 a² + 0.31 a + 0.38/a² s with a = K/600. That predicts K899 at about 151 s [142, 165], about 1.10x. K899 is the largest K whose folded root table keeps 2^27 16-byte slots (2 GiB); K900 needs 2^28. The nested base `n83-compact-orbit-k899-20261012/runs/K899/base.jsonl` holds B's K600 base as an exact prefix: the same 600 representatives, 99,600 points and labels.

Does K899 lower the **end-to-end cold** cost of the rank-probe producer, against K600, on new targets and new rank seeds, with every answer verified and the factor-log table of each K verified? This is an iteration under AGENTS.md "Success criterion". A promoted result becomes the new n = 83 row.

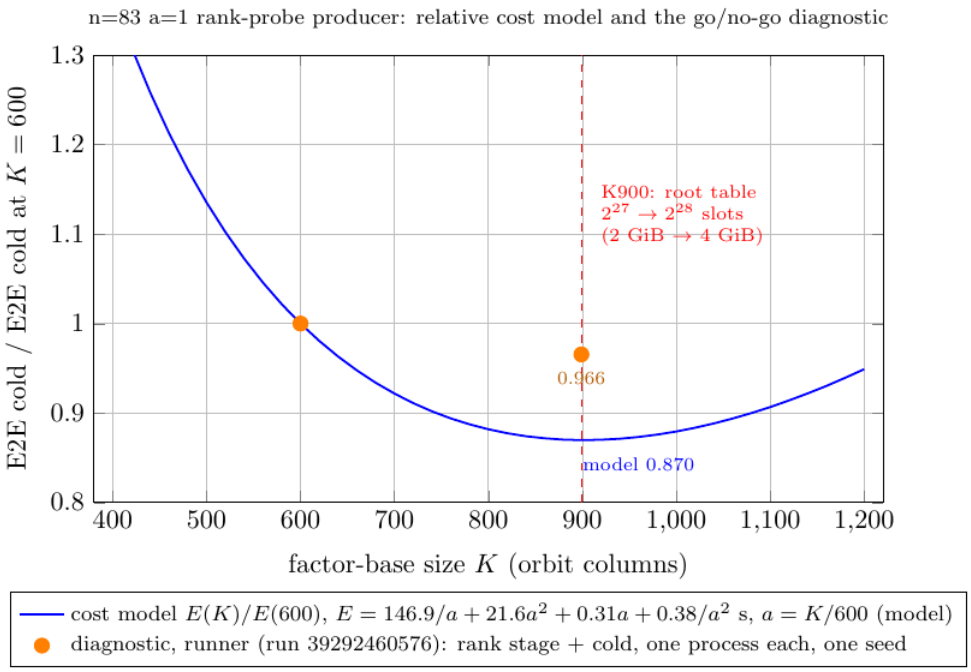


Figure 1: Relative cost: the cost model (blue, a model) and the runner diagnostic (orange, one process per arm). Source: figures/design.tex, rendered by `make_figures.py`; vector: figures/design.pdf.

Does the producer support K other than 600? (yes, with no code change)

The producer is the rank-probe producer, unchanged. `restore.py` derives it with the rank-probe experiment's own `make_online_source.py` and five parents, verbatim, and checks the frozen digest `d4394df3...`. K is an input: the base file. Checked in the source (`snapshots/r770/examples/koblitz_orbit_dlp_fast_online_rankprobe.rs`):

- **Hard-coded 600s.** They appear only in two unit tests: the Feistel bijection on 29,880,000 states, and `Fold128::new(600, 83)` against 14,940,300 stored states. Neither runs in a process.
- **Slot capacity.** The table is `CompactRootTable128::with_capacity(2xstates)`, so it has (4 x stored states).next_power_of_two() slots: 2^26 at K600 and 2^27 at K899 (4 x 33,540,791 = 134,163,164 < 2^27).
- **Packed fields.** The 16-byte slot stores the four `pack` fields in 11 bits each, so K < 2048 is fine. Keys are 83 bits.
- **Folded-index sizes.** `Fold128::new(columns, n)` takes them from the base. The echelon, the rank target (full rank = columns) and the factor-log table width come from the base's orbit columns too.
- **What was K600-specific.** Only the experiment wrapper. The rank-probe `arm.py` hard-codes the stored states, the slot count, the table entries and B's index digests. Here `arm.py` reads them per arm from `protocol.json online.k.<K>`.

The runner diagnostic confirms this. At K899: 899 columns, 67,080,683 logical and 33,540,791 stored states, 2^27 slots (2,147,483,648 bytes), 67,077,932 entries, full rank in 899 attempts with 0 failures and 0 rows without gain, peak RSS 3.88 GB (K600: 1.88 GB).

Go/no-go diagnostic (runner, run 39292460576)

The host was an AMD EPYC 9V45 GitHub runner with transparent huge pages `madvise`. That is a different and faster host class than run 39214027289's (K600 rank stage 105.6 against 141.5 ns per probe). One pinned CPU. Order: cP600, kP899, kP600, cP899, cP600, cP899, cPI600, cPI899. Rank seed 8991, used by no frozen run. Source: diag/run_diag.py, diag/run_github_39292460576/diag.json.

	K600	K899	K899 / K600	gate
index build, cold (mean of 2)	15.70 s	**37.92 s**	2.42	K899 $\leq$ 55 s: **pass**
index build, lift_scan (cPI)	12.74 s	31.07 s	2.44	
setup, cold (mean of 2)	17.23 s	39.01 s	2.26	
cold (setup + online, mean of 2)	17.56 s	39.12 s	2.23	
rank stage	111.71 s	85.70 s	0.767	
rank ns per probe, all attempts	105.6	**121.5**	1.150	K899 $\leq$ 150 ns: **pass**
rank ns per probe, first 50 attempts	103.7	**120.0**	1.157	K899 $\leq$ 150 ns: **pass**
first 50 attempts	10.23 s	4.46 s		
mean probes per attempt / law	1.019	1.016		
online ns per probe (S3 loop, Q0)	158.7–171.3	181.1–238.7		
single-process E2E (rank + cold)	129.3 s	124.8 s	**0.966**	

Verdict: GO. Both thresholds are met on the runner. The per-probe cost rose 15%, though, where the cost model assumed 1.0. That is the 2 GiB table, and it moves the predicted end-to-end ratio from about 1.10 to **about 1.036** (this host, one seed).

Local (cloud-agent VM, contended, THP `madvise`): the per-probe ratio K899/K600 was 1.22 (405 against 496 ns, with glibc's `glibc.malloc.hugetlb=1` tunable, back to back) and 2.56 under heavier load. The index ratio was 2.0–2.5. These are not used. The VM inflates lookups into the 2 GiB table, as expected, and the runner is the instrument. The local records are in `smoke/`.

Exactness at K899

K899 cannot reproduce K600's probes, since it is a different index. Exactness here means four things, each a frozen gate:

1. **The factor-log table is self-consistent and verified.** The producer checks [L_j]G = R_j for every column before any target. `check_log_table.py` re-checks every log of both frozen tables in the pinned pure-Python oracle in every Tier 1 job. The K899 table (`inputs/factor_logs_n83_K899.json`, logs sha256 `c6652350...`) was written by the diagnostic's rank stage. It passed the oracle on all 899 columns on the runner and locally; the two runs wrote the same logs. The K600 base is an exact prefix of the K899 base, and discrete logs of fixed points are unique mod r, so the first 600 K899 logs must equal B's K600 table, and they do. Every Tier 1 rank process must write its K's table log for log (B's at K600).
2. **Same scalar everywhere.** For every target, the scalar is the same in every K600 process, every K899 process and rT, and it replays as [d]G = Q in the oracle. At Q0, K899 gives scalar 3141592653589793 with relation [69918, 46431, 52870, 61057] and 486,779 probes. K600 gives 1,987,852 probes with B's archived relation. Both are frozen identities.
3. **Within K899, the rank arms agree attempt for attempt.** kP899, kP899aa and kPX899 must give one relation digest, one per-attempt probe vector, one root-call total and one set of point-addition counters per block. The planted arm kPX899 also checks the frame keys against the rank-cut keys on every candidate at K899.

4. **The probe law holds.** Per K, the mean probes per rank attempt pooled over the four blocks, divided by r / (166 E_K), must fall in **[0.90, 1.10]**. This band was frozen before any timing. With 2,400 (K600) and 3,596 (K899) attempts the standard error is about 2.0% and 1.6% (CV 0.97 per attempt). The archived Feistel-order K600 ratios were 0.978 and 1.036, and the diagnostic gave 1.019 and 1.016. ±10% is about five standard errors.

Frozen Tier 1 design

One GitHub `ubuntu-24.04` runner, one pinned CPU, Tier 1 counters, one fresh process per run (`.github/workflows/n83-k899.yml`). Every IC arm runs `ic_online_rankprobe_r770` with lift_scan target keys, the Feistel order, prefetch, and the 16-byte compact table over the folded index.

Arm	Kind	K	Role
`cP600`	cold	600	**baseline** cold (the rank-probe producer's cP)
`cP899`	cold	899	**candidate** cold
`cP899aa`	cold	899	A/A: a second cold process of cP899
`cPI600`, `cPI899`	cold	600, 899	pre-registered secondaries: lift_scan in the index build (`KIC_INDEX_CANON=lift`); never decide
`rT`	rho		tuned rho rT770, same points
`kP600`	rank	600	**baseline** rank stage (`KIC_RANK_SOLVE=frame`)
`kP899`	rank	899	**candidate** rank stage
`kP899aa`	rank	899	A/A: a second process of kP899
`kPX899`	rank	899	planted positive control: `frame_plant` at K899 (redoes the rank-cut key derivation on every candidate and compares; a known extra cost, about 1.97x per probe in run 39214027289)

Baseline choice. The index-lift run (`claude/n83-indexlift`, run 39290913632) had not landed at freeze time. So the baseline is the rank-probe producer, and the lift_scan index build runs as the cPI pair at both K.

- **Panel.** 20 new public points, role `k899-cold-v1` (inputs/cold_targets.json, `make_targets.py`). They are disjoint from Q0, B's panel and every earlier n = 83 panel of B's law. That includes `rankprobe-cold-v1` (which the rotation-key draft reuses) and `indexlift-cold-v1`. It was checked against the committed panel files too.
- **Rank blocks.** Four new blocks K1–K4, rank seeds 401–404, on units s000–s003. Every rank arm of a block runs the same seed, so K600 and K899 draw the same scalars [a]G for their rank targets. Cold arms are paired by point.
- **Order.** O1 (Q0), then C1–C5, K1, C6–C10, K2, ... K4. Arms are shuffled within each segment with fixed seeds.

Frozen rule

- **Primary.** R = E2E(kP600, cP600) / E2E(kP899, cP899). Here E2E(k, c) is the mean of k's `timing_ms.rank_stage` over the rank blocks plus the mean of c's `cold_in_process_ms` over the 20 points, so it covers setup, index build, rank stage, linear algebra and one-target online.

- **Bootstrap.** 10,000 draws. The rank blocks are resampled with the same drawn ids for both arms. Within each drawn block, each arm's attempts are resampled independently: the two K run different attempts, so they cannot be paired. Each process keeps its non-attempt time. The cold points are resampled, paired across cP600 and cP899.
- **Decision.** Improvement iff the 95% lower bound of R is above 1. Regression iff the upper bound is below 1. Otherwise not separated.
- **Invalid.** The verdict is invalid if any gate or control fails, or any scheduled row is missing (time-guard skips included).
- **Controls.** A/A: E2E(kP899, cP899) / E2E(kP899aa, cP899aa), attempts paired, and cold cP899 / cP899aa must both be inside [0.97, 1.03]. Planted: kPX899 / kP899 per attempt (paired) must have its 95% lower bound above 1.
- **Gates.** As listed in the protocol: per-K layout and digests in `arm.py`; one scalar per point plus oracle replay; one relation and one probe count per point and K; the Q0 identities; the rank tables; the in-block K899 agreement; the frame counters; the law band.
- **Secondaries** never decide. They are: the whole rank stage kP600 / kP899; ns per probe (all attempts and the first 50); cold, setup, index, online and RSS ratios; the cPI pair; and S and the gaps to rT.
- **Time.** Job limit 350 min, guard 300 min. The diagnostic's process walls with a 1.5x margin give segment upper estimates of C 232 s, K 881 s and O 88 s: 157.5 min with a 20-min build allowance, 111.4 min expected. On run 39214027289's slower host class (about 1.34x per probe) the expected total is about 150 min.

Power (before freezing)

Source: `power_check.py` on `inputs/power_prior.json`. The prior is run 39214027289's kP / kPaa attempts (seeds 301–304) and its cP / cR / cPI cold. A simulated K899 block draws 899 K600 attempts rescaled: the probes-to-first-hit are geometric, so the shape of the per-attempt distribution is kept and only its mean moves. Its cold is cR times the diagnostic's cold ratio, 2.227. The frozen bootstrap is applied, with 400 simulations of 400 draws (`analysis/power.json`).

end-to-end effect R	1.00	1.03	1.036 (diagnostic)	1.05	1.07	1.10
power, 4 blocks (the design)	0.018	0.14	**0.16**	0.375	0.65	**0.91**
8 blocks (information only)	0.005		0.34	0.63		
12 blocks (information only)	0.005		0.48	0.81		

Reading. The interval is set by the rank stage's own randomness, not by timing noise. Probes per attempt are geometric (CV 0.97), and the two K run different attempts. Four blocks resolve the model's 1.10 but not the diagnostic's 1.036. If the runner's 15% per-probe penalty holds, a `not_separated` verdict is the likely outcome. The design keeps the four blocks it was asked for.

Files

File	Role
`protocol.json`, `schedule.json`	frozen protocol and run order (`make_protocol.py`, `make_schedule.py --check`)
`restore.py`, `make_online_source.py` + five parents, `online/`, `lift2/`	the unchanged rank-probe producer, digest-checked
`arm.py`, `run_schedule.py`	one arm per process, with K-specific gates; the pinned schedule runner

File	Role
verify_receipt.py	gates, controls, primary and secondaries
check_log_table.py	both tables in the oracle, the prefix and the first 600 logs
inputs/k899_identity.json	frozen K899 facts and the diagnostic summary (diag/make_identity.py)
inputs/factor_logs_n83_K899.json	the K899 factor-log table
diag/	the go/no-go diagnostic script, workflow .github/workflows/n83-k899-diag.yml, archived runner output
smoke/	local records: the contended VM diagnostic, the THP rank pair, the O1 smoke through run_schedule.py (correctness only)
make_figures.py, figures/	design figure; make_figures.pyresult<result.json> after the run
README.pdf	this report: pandocREADME.md-oREADME.pdf--pdf-engine=pdflatex-Hreport_header.tex--lua-filterreport_filter.lua-Vgeometry:margin=2cm-Vfontsize=10pt-Vcolorlinks=true